

Búsqueda de factores RSA compartidos

en los registros de Transparencia de Certificados

Un enfoque basado en el algoritmo Binary Tree Batch GCD

David Quinzanos Saiz

Codirectores: Ana I. Gómez Pérez · Domingo Gómez Pérez

Máster Interuniversitario (UC–UIMP) en Data Science · Junio de 2026

Introducción y motivación
Estado del arte
Metodología

Resultados
Conclusiones

- La seguridad en Internet se apoya en **certificados digitales** firmados por autoridades certificadoras.
- **Certificate Transparency (CT)**: las CAs deben publicar los certificados en *logs* públicos, verificables y de solo anexo.
- Gran parte de esos certificados emplea **RSA**, cuya seguridad depende de factorizar $N = p \cdot q$.
- Los CT logs son una **enorme fuente de claves públicas reales** para auditar la criptografía desplegada.

La grieta

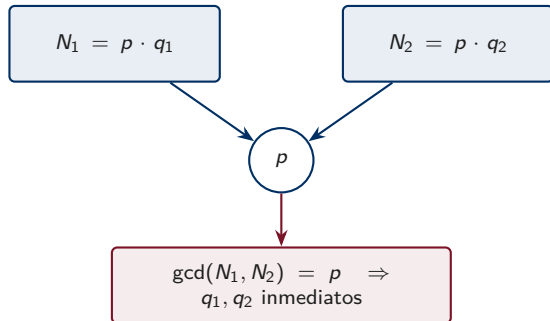
La seguridad de RSA no solo depende de las matemáticas, sino de **cómo se generan las claves** en la práctica.

El problema: factores primos compartidos

- Con **baja entropía**, dos claves distintas pueden **reutilizar un primo**.
- Si $N_1 = p \cdot q_1$ y $N_2 = p \cdot q_2$, entonces

$$\gcd(N_1, N_2) = p$$

- Recuperar p **rompe ambas claves** al instante, sin factorizar.
- En condiciones ideales esto es prácticamente imposible: si ocurre, hubo un **fallo de generación**.



Objetivo general

Estudiar la **detección eficiente de factores primos compartidos** entre módulos RSA y aplicarla a certificados reales de Certificate Transparency.

- Revisar los fundamentos de la vulnerabilidad y los métodos *batch GCD*.
- Implementar el **Binary Tree Batch GCD** (Pelofske, 2024) en **C++17/GMP**.
- Comparar su rendimiento con las implementaciones Python de referencia.
- Construir un flujo para **extraer, normalizar, validar y deduplicar** módulos reales de ct-archive y auditarlos.

- Heninger et al. (2012): millones de claves de HTTPS/SSH; muchas factorizables por factores compartidos.
- La línea continúa con análisis cada vez más masivos:

Trabajo	Claves analizadas	Factorizadas
Hastings et al. (2016)	81 M	>313.000
Kilgallin & Vasko (2019)	~57 M	249.548
Våge (2022) — <i>CT logs</i>	159,4 M (RSA-2048)	8 (+355 ZeroSSL)

Våge concluye que solo se ha auditado una **pequeña fracción** de los CT logs: queda margen para análisis mayores $\Rightarrow$ hacen falta algoritmos más eficientes.

Problema *all-to-all* GCD

- Comparar todos los pares es **cuadrático**: $\binom{M}{2}$.
- **Clásico**: *product tree* + *remainder tree*.
- Cuello de botella: enteros gigantes en la cima del árbol $\Rightarrow$ tiempo y **memoria**.

Binary Tree Batch GCD (Pelofske, 2024)

- **Evita construir el remainder tree**.
- Misma complejidad asintótica, mejor coste práctico ($\approx 6\times$ en el artículo).
- Base algorítmica de este TFM.

Encaje del trabajo

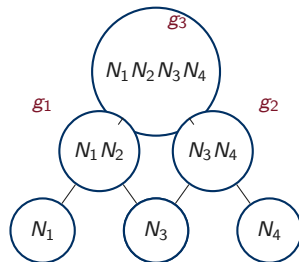
Partimos del **contexto experimental de Våge** (CT logs) y sustituimos el núcleo algorítmico por la **propuesta de Pelofske**, con una implementación propia en C++/GMP.

Binary Tree Batch GCD en tres fases

1. **Árbol de productos** con $\gcd$ en línea: por cada par (A, C) se calcula $A \cdot C$ y $\gcd(A, C)$.
2. **Agregación:** $B = \prod_{j=1}^{M-1} g_j$.
3. **Enumeración:** para cada N_i , $\gcd(N_i, B)$ no trivial revela un factor.

Coste

Solo $2M - 1$ operaciones de $\gcd$ frente al número cuadrático del enfoque ingenuo.



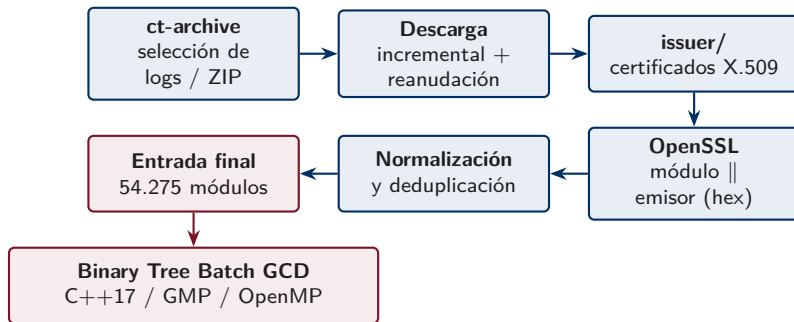
$B = g_1 g_2 g_3$
(líneas: productos $\uparrow$,
 $\gcd$ entre hermanos)

- **C++17 + GMP**: aritmética multiprecisión y control fino de memoria.
- **Liberación temprana** de los hijos de cada nodo $\Rightarrow$ menor pico de RAM.
- **OpenMP** opcional en la enumeración final (hasta 18 hilos).
- Referencias Python de Pelofske *sin modificar* su lógica (vía *wrappers*).

Validación funcional

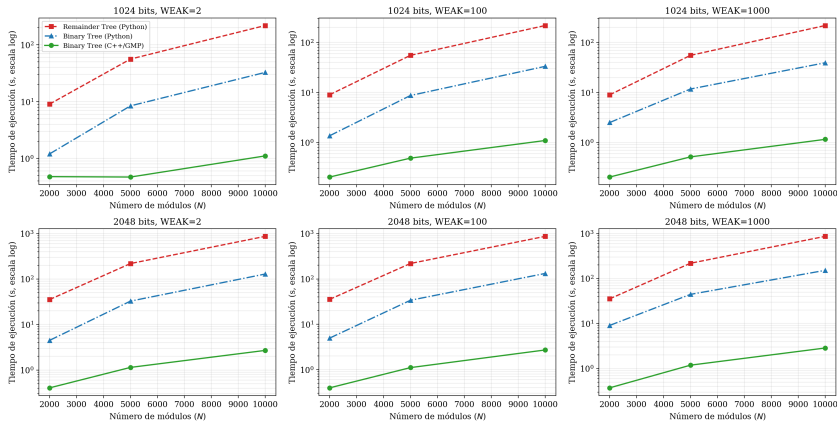
Las **tres** implementaciones detectan **exactamente los mismos** módulos vulnerables en cada configuración, y se verifica $N \bmod p = 0$.

Flujo de datos reales desde ct-archive



Foco en issuer/ (CAs e intermediarias): subconjunto manejable que conserva la relación módulo || emisor.

Comparativa sintética: tiempos de ejecución



Réplica reducida del experimento de Pelofske (RSA-1024/2048; WEAK = 2, 100, 1000).

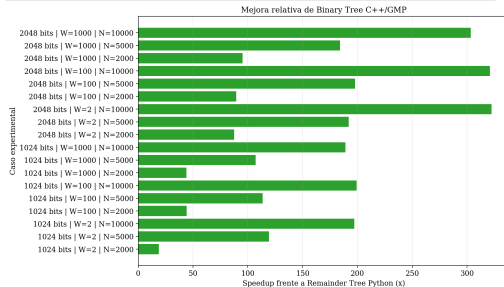
Escala logarítmica: el orden **Remainder** > **Binary Python** > **C++/GMP** se mantiene en todas las configuraciones.

Aceleraciones (media / máx)

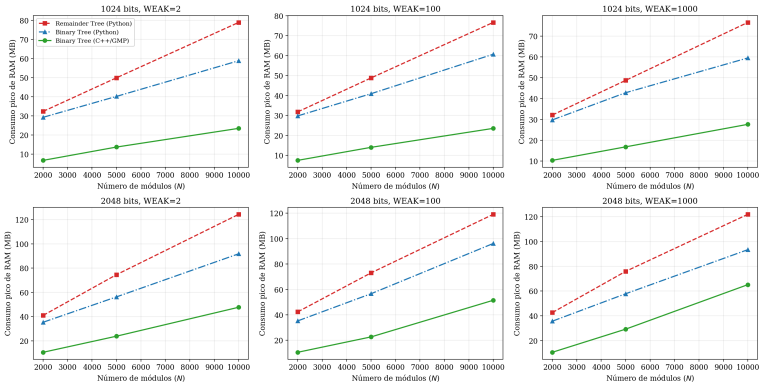
- Binary Python vs. Remainder: **6,1×** (3,6–7,9)
- C++/GMP vs. Remainder: **116,5×** (máx 275,3×
- C++/GMP vs. Binary Python: **18,7×** (máx 41,3×

Caso 2048b, $N=10000$, WEAK 1000

Remainder Py	Binary Py	C++/GMP
866,8 s	152,8 s	5,85 s



Comparativa sintética: consumo de memoria

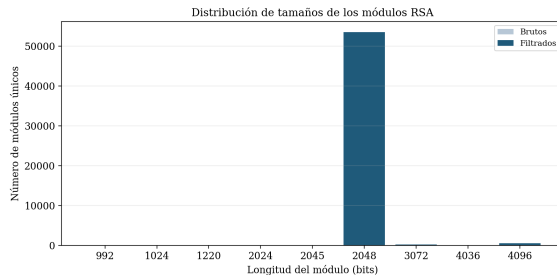


Pico de RAM: **128,7 MB** (clásico) · **94,4 MB** (Binary Py) · **57,9 MB** (C++/GMP).
Reducción del **55,0 %** frente al clásico y del **38,7 %** frente a Binary Python, gracias a la liberación temprana de nodos.

Resultados sobre datos reales (CT logs)

Elemento	Valor
Módulos únicos brutos	54.278
Módulos RSA plausibles	54.275
Descartados (validación)	3
Logs resumidos	20
GCDs no triviales	0
Claves vulnerables	0
Tiempo total	≈23,8 s

Auditoría completa en segundos, frente a los *minutos* de las referencias.



Mayoría de claves RSA-2048 (53.539).

1. El **Binary Tree Batch GCD** es una alternativa práctica al enfoque clásico: reduce de forma consistente el tiempo de ejecución.
2. La **implementación C++17/GMP** mejora de forma decisiva a las versiones Python (hasta $\sim 275\times$ en tiempo y -55% en memoria).
3. El comportamiento frente a WEAK encaja con la teoría: ventaja máxima cuando los factores compartidos son **poco frecuentes** (el caso real).
4. Se entrega un **flujo completo y reproducible**: extracción, validación, deduplicación y auditoría de módulos reales de CT logs.

Resultado sobre datos reales

No se detectaron factores compartidos entre los 54.275 módulos analizados; un resultado **acotado** al subconjunto extraído, no una garantía global.

Limitaciones

- Parrilla sintética reducida ($N \leq 10000$) por recursos locales.
- Una medición por configuración (no estadística).
- Conjunto real limitado a certificados de issuer/.
- Agregación de B aún secuencial.

Trabajo futuro

- Incorporar **certificados hoja** de las entradas completas.
- Paralelizar fases aún secuenciales.
- Escalar a conjuntos cercanos a la literatura (>100 M).

¡Gracias!

¿Preguntas?